Testing Email

When we used the generate script to create our order mailer, it automatically constructed a corresponding `order_test.rb` file in the application's `test/mailers` directory. It's pretty straightforward; it simply calls each action and verifies selected portions of the email produced. Because we've tailored the email, let's update the test case to match:

rails80/depot_qa/test/mailers/order_mailer_test.rb

```
    require "test_helper"

    class OrderMailerTest < ActionMailer::TestCase
      test "received" do
»       mail = OrderMailer.received(orders(:one))
»       assert_equal "Pragmatic Store Order Confirmation", mail.subject
»       assert_equal [ "dave@example.org" ], mail.to
»       assert_equal [ "depot@example.com" ], mail.from
»       assert_match /1 x The Pragmatic Programmer/, mail.body.encoded
      end

      test "shipped" do
»       mail = OrderMailer.shipped(orders(:one))
»       assert_equal "Pragmatic Store Order Shipped", mail.subject
»       assert_equal [ "dave@example.org" ], mail.to
»       assert_equal [ "depot@example.com" ], mail.from
»       assert_match %r{
»         <td[^>]*>1<\/td>\s*
»         <td>&times;<\/td>\s*
»         <td[^>]*>\s*The\sPragmatic\sProgrammer\s*</td>
»       }x, mail.body.encoded
      end
    end
```

The test method instructs the mail class to create (but not to send) an email, and we use assertions to verify that the dynamic content is what we expect. Note the use of `assert_match` to validate just part of the body content. Your results may differ depending on how you tailored the `default :from` line in your `OrderMailer`.